2

CS 61B Fall 2025 — Final Thursday, December 18, 2025

PRINT Your Name: Adithya Giri

PRINT Your Student ID: 3041015344

PRINT Student name to your left: Alex Toohey

PRINT Student name to your right: Anishka Banerjee

You have 170 minutes. There are 8 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	7	8	Total
Points:	6	7	11	12	21	25	3	15	100

For questions with **circular bubbles**, select only one choice (there is only one correct answer).

- ○ Unselected option (completely unfilled)
- ⊘ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square boxes**, you may select one or more choices (select all that apply).

- ■ You can select
- ■ multiple squares
- ☑ Don't do this (it will be graded as incorrect)

- Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers or your answer is ambiguous, we will grade the **worst** interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) or a Java library not on the reference card (e.g. Google Truth) is relevant, it will be explicitly noted in the question.
- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B/61BL style guidelines. For example, any method, constructor, or if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Read the honor code below and sign your name.

> By signing below, I affirm that all work on this exam is my own work. I have not referenced any disallowed materials, nor collaborated with anyone else on this exam. I understand that if I cheat on the exam, I may face the penalty of an "F" grade and a referral to the Center for Student Conduct.

SIGN your name: Adithya

02137

Q1 *Welcome to 61B* (6 points)

Q1.1 (1 point) True or False: The following code compiles and runs.

```
List<Integer> L = new List<>();
```

○ True ● False

For Q1.2 and Q1.3, suppose we run the following code:

```
HashMap<Integer, String> gruArmy = new HashMap<>();
gruArmy.put(1, "kevin");
gruArmy.put(2, "stuart");
gruArmy.put(1, "bob");
```

Q1.2 (1 point) What does `gruArmy.size()` return?

○ 1 ● 2 ○ 3

Q1.3 (1 point) What does `gruArmy.get(1)` return?

○ `"kevin"` ○ `"stuart"` ● `"bob"` ○ `List.of("kevin", "bob")`

Q1.4 (3 points) Which statements must be true for the following code to compile? Select all that apply.

```
JeopardyBoard jb = new JeopardyBoard(49);
for (Tile t : jb) {
    int k = t.compareTo(t);
}
```

- [x] The `JeopardyBoard` class must implement the `Iterable` interface.
- [ ] The `JeopardyBoard` class must implement the `Iterator` interface.
- [ ] The `Tile` class must implement the `Iterable` interface.
- [ ] The `Tile` class must implement the `Iterator` interface.
- [ ] The `Tile` class must implement the `compareTo` method.
- [x] The `Tile` class must implement the `Comparable` interface.
- [ ] The `Tile` class must implement the `Comparator` interface.

○ None of the above

Q1.5 (0 points) **This question is worth zero points and is just for fun!** In Greek Mythology, the titan Atlas is tasked with holding up the sky. Name a figure from another mythological tradition whose hidden work is necessary for the proper functioning of the universe.

In Beeland, the mythical figures of Hosh Jug and Keyrin Pao are p cool.

Q2 *Implementation Inheritance [Midterm 1 Redemption]* (7 points)

Your score on this problem will be directly added to your Midterm 1 score.

Suppose we have a class called `MaxStringList` which `extends ArrayList<String>`. It has only one method `maxString()`, which returns the string in the list that is last in alphabetical order.

Reminder from front page of exam: You may not need all lines, and your answer will be reformatted.

Q2.1 (3 points) Write a Google Truth unit test for this method. Do not worry about testing edge cases.

Syntax for testing: `assertThat(a).isEqualTo(b);`

```
@Test
public void testMaxString() {
    MaxStringList msl = new MaxStringList();
    msl.add("abba");
    msl.add("zebra");
    assertThat(msl.maxString()).isEqualTo("zebra");

}
```

Q2.2 (4 points) Implement the `maxString` method of `MaxStringList extends ArrayList<String>`.

You may assume that the list has at least one item, no duplicate items, and no `null`s. You may not use `Collections.max`. You may not modify the list. The method must complete in $\Theta(N)$ time.

```
public String maxString() {
    String maxString = get(0);
    for (int i = 1; i < size(); i++) {
        if (maxString.compareTo(get(i)) > 0) {
            maxString = get(i);
        }
    }
    return maxString;

}
```

Q3 *Clubs 'n Countries [Midterm 1 Redemption]* (11 points)

Your score on this problem will be directly added to your Midterm 1 score.

Avik wants to keep track of the students in UC Berkeley's clubs. Each club is represented by the `Club` class below, which maps every student in that club to their home country.

```
public class Club {
    public Map<Student, Country> countryMap;
    ...
}

public class Student {
  /* Implementation omitted */
}
public class Country {
  /* Implementation omitted */
}
```

On the next page, implement `countByCountry`, which takes in a list of `Clubs`, and returns a map from each `Country` to the number of unique students from that country. The map should only contain countries that appear in the `countryMaps`.

If a `Student` is in multiple clubs, then each of those clubs will map that student to the same `Country`. Make sure to avoid counting the same `Student` twice if they are in multiple clubs.

You may assume that there is at least one club, and each club has at least one student.

Here is an example with 2 clubs and 3 total students:

Club	**Country Map**
Chess Club	{ Aditya: Scotland, Natalia: Brazil, Rushil: Scotland }
Climbing Club	{ Natalia: Brazil }

`countByCountry` should return the following map: { Brazil: 1, Scotland: 2 }.

Reminder from front page of exam: You may not need all lines, and your answer will be reformatted.

Syntax hint: `someMap.keySet()` returns a `Set` of all keys in this map.

```
public static Map<Country, Integer> countByCountry(List<Club> allClubs) {
    Map<Country, Integer> counts = new HashMap<>();
    HashSet<Student> dupeChecker = new HashSet<>();
    for (Club club : allClubs) {
        for (Student s : club.countryMap.keySet()) {
            Country c = club.countryMap.get(s);
            if (!dupeChecker.contains(s)) {
                if (!counts.containsKey(c)) {
                    counts.put(c, 0);
                }
                counts.put(c, counts.get(c)+1);
                dupeChecker.add(s);
            }
        }
    }
    return counts;
}
```

Q4 *Taste the Rainbow! [Midterm 1 Redemption]* (12 points)

Your score on this problem will be directly added to your Midterm 1 score.

After eating too many Skittles, Karen adds a `rainbowfy()` method to the `LinkedListDeque` from Project 1. `rainbowfy` reorders the items in the deque into a *rainbow order.*

A rainbow order is defined by picking items from the original order, alternating between the back and the front. For example:

- The rainbow order of **[1, 2, 3, 4, 5, 6, 7, 8]** is **[8, 1, 7, 2, 6, 3, 5, 4]**.
- The rainbow order of **[1, 2, 3, 4, 5]** is **[5, 1, 4, 2, 3]**.

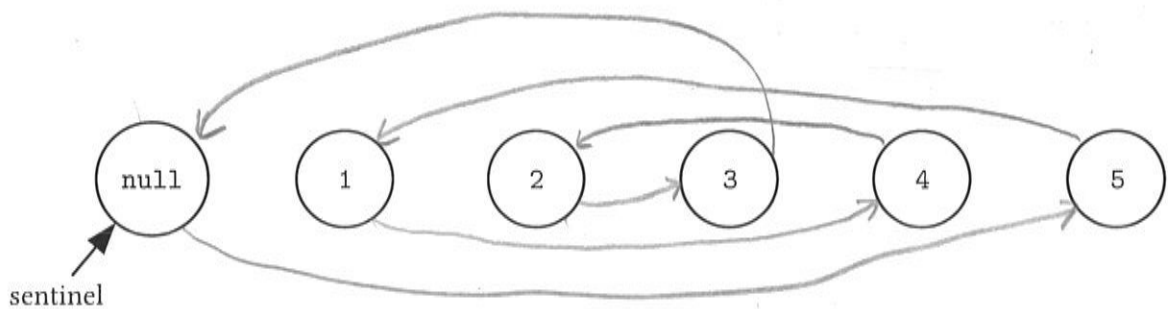
Karen splits `rainbowfy` into two helper functions, `rainbowfyNexts` and `rainbowfyPrevs`. The two functions' implementations **do not** depend on each other.

Below is the `LinkedListDeque` implementation, with `rainbowfy` added:

```
public class LinkedListDeque<T> {
    Node sentinel;

    private class Node {
        Node prev;
        T item;
        Node next;
        ...
    }
    ...

    public void rainbowfy() {
        rainbowfyNexts();
        rainbowfyPrevs();
    }

    private void rainbowfyNexts() { /* Implemented in Q4.2 */ }
    private void rainbowfyPrevs() { /* Implemented in Q4.3 */ }
}
```

Q4.1 (1 point) Given a deque initially containing [1, 2, 3, 4, 5], draw arrows to represent the `next` pointers after `rainbowfy()` modifies the deque. *Hint: Make sure you draw the right number of arrows!*

Do not draw any `prev` pointers.



Reminder from front page of exam: You may not need all lines, and your answer will be reformatted.

For Q4.2 and Q4.3, you **may not** use the `addFirst`, `addLast`, `removeFirst`, or `removeLast` methods. You may assume that the deque has `size` ≥ 2.

Q4.2 (6 points) Implement `rainbowfyNexts`, which reassigns only the `next` pointers of all nodes (not any `prev` pointers). *Hint: Use the diagram you drew for a visualization!*

```
private void rainbowfyNexts() {

    Node left = sentinel;
    Node right = sentinel.prev;

    for (int i = 0; i < this.size(); i += 1) {

        if (i % 2 == 0) {
            right.next= left.next;
            left.next= right;
            right.prev.next= right.next;

        } else {
            right= right.prev;
            left= right.next;
        }
    }
    right.next= sentinel;
}
```

Q4.3 (5 points) Implement `rainbowfyPrevs`, which reassigns only the `prev` pointers of all nodes (not any `next` pointers). You may assume `rainbowfyNexts` is correctly implemented and is called immediately before `rainbowfyPrevs`.

```
private void rainbowfyPrevs() {
    Node curr = sentinel;
    for (int i = 0; i < this.size(); i += 1) {
        curr.next.prev = curr;
        curr = curr.next;
    }
}
```

Q5 *Potpourri* (21 points)

Q5.1 (2 points) We want to insert N unique integers into an initially empty **binary search tree** (BST). Which statement is true?

- ○ Any insertion order takes $O(N \log N)$ time.
- ● There exists an insertion order that takes $O(N \log N)$ time, and finding this insertion order **can** always be done in $O(N \log N)$ time.
- ○ There exists an insertion order that takes $O(N \log N)$ time, but finding this insertion order **cannot** always be done in $O(N \log N)$ time.
- ○ There does not exist an insertion order that takes $O(N \log N)$ time.

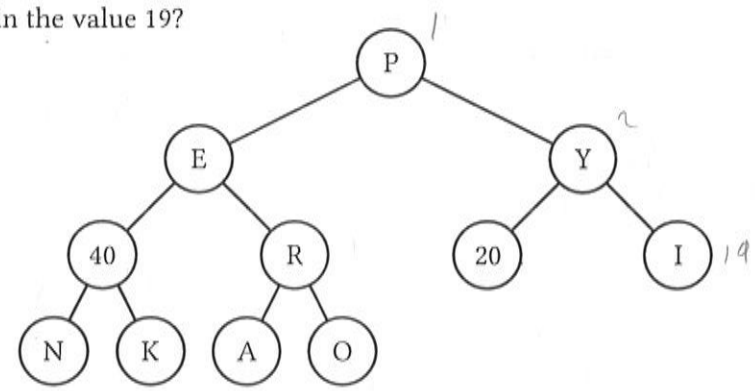
Q5.2 (2 points) If we perform a `rotateLeft` operation on a BST, which of the following traversals can change as a result of the rotation?

■ Pre-order ☐ In-order ■ Post-order ○ None of these

Q5.3 (2 points) Suppose we have a perfectly balanced (bushy) BST with hundreds of nodes. If we perform a BFS from the root, which of the following traversals could yield the same visit order as the BFS?

☐ Pre-order ☐ In-order ☐ Post-order ● None of these

Q5.4 (2 points) Consider the **min heap** below, which contains **unique integers**. We know the values in two of the nodes (20 and 40), and the letters in the other nodes represent unknown values. Which nodes could contain the value 19?



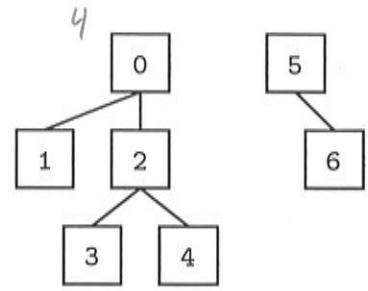
☐ P ■ E ■ Y ■ R ■ I ☐ N ☐ K ■ A ■ O

Q5.5 (2 points) Select all true statements about Weighted Quick Union **without** path compression.

- ■ The runtime of `union` is $O(\log N)$.
- ☐ The runtime of `union` is $O(N)$.
- ☐ The runtime of `isConnected` is $O(1)$.
- ■ The runtime of `isConnected` is $O(\log N)$.
- ○ None of the above

Q5.6 (2 points) Consider the Weighted Quick Union **with** path compression below.

What is the parents array after we call `isConnected(3, 5)`?



- ○ [−5, 0, 0, 2, 0, −1, 5]
- ○ [−5, 0, 0, 2, 2, −2, 5]
- ● [−5, 0, 0, 0, 2, −2, 5]
- ○ [1, 2, 3, 4, 5, 6, 7]
- ○ [−5, 0, 0, 2, 2, 0, 0]
- ○ [5, 5, 5, 5, 5, −7, 5]
- ○ [5, 0, 0, 0, 2, 2, 5]
- ○ [−7, 0, 0, 0, 0, 0, 0]

Q5.7 (2 points) What is the worst-case runtime to add N strings to a hash table set with linked list buckets?

○ $\Theta(1)$ ● $\Theta(N)$ ○ $\Theta(N \log N)$ ○ $\Theta(N^2)$ ○ None of these

For Q5.8 to Q5.10, consider a `Trie t` that stores the set of strings `["a", "apes", "in", "inch", "into"]`. The subparts are **independent**.

Q5.8 (1 point) For which choices of `key` will `t.contains(key)` return true? Select all that apply.

☐ `""` ☒ `"a"` ☒ `"apes"` ☐ `"inc"` ☐ `"inches"`

Q5.9 (1 point) If we call `t.add("ape")`, how many new nodes would be created?

● 0 ○ 1 ○ 2 ○ 3 ○ 4 or more

Q5.10 (1 point) If we call `t.add("bee")`, how many new nodes would be created?

○ 0 ○ 1 ○ 2 ● 3 ○ 4 or more

For Q5.11 and Q5.12, consider the unsorted array: `[6, 3, 2, 7, 5, 1, 4, 8, 0, 9]`. Each cell below has the intermediate array at a step for some sort that has not yet completed.

A:	`[5, 3, 4, 2, 0, 1, 6, 7, 8, 9]`	B:	`[2, 1, 0, 3, 5, 4, 6, 7, 8, 9]`
C:	`[3, 6, 2, 5, 7, 1, 4, 0, 8, 9]`	D:	`[1, 2, 3, 5, 6, 7, 4, 8, 0, 9]`

Q5.11 (2 points) Which array is an intermediate step in Quicksort where we use the leftmost element as pivot? Assume we do not shuffle. Assume we use the 3-scan partitioning strategy.

○ A ● B ○ C ○ D

Q5.12 (2 points) Which array is an intermediate step in Heapsort, using bottom-up heapification and a max heap?

● A ○ B ○ C ○ D

Q6 *Midterm 2 Nostalgia* (25 points)

Q6.1 (3 points) What is the runtime of `rushil` in terms of N?

```
public void rushil(int N) {
    if (N == 0) {
        return;
    }
    rushil(N/3);
    linearPhoenix(N); // runs in Θ(N) time
    rushil(N/3);
    linearPhoenix(N); // runs in Θ(N) time
    rushil(N/3);
}
```

$2N$

$\frac{2}{3}N \quad \frac{2}{3}N \quad \frac{2}{3}N \rightarrow 2N \Big\} \log_3 N$

$\frac{2}{9}N$

$N \log N$

○ $\Theta(1)$	○ $\Theta(\log(\log N))$	○ $\Theta(\log N)$	○ $\Theta((\log N)^2)$	○ $\Theta(\sqrt{N})$
○ $\Theta(N)$	● $\Theta(N \log N)$	○ $\Theta(N^2)$	○ $\Theta(N^2 \log N)$	○ $\Theta(N^3)$
○ $\Theta(N^3 \log N)$	○ $\Theta(2^N)$	○ $\Theta(N!)$	○ $\Theta(N^N)$	○ Infinite loop

Q6.2 (3 points) What is the runtime of `lucky` in terms of N, where N is the length of `arr`?

```
public void lucky(int[] arr) {
    for (int i = arr.length - 1; i >= 0; i -= 1) {
        for (int j = 0; j < Math.pow(2, i); j += 1) {
            arr[i] += arr[i];
        }
    }
}
```

$N \rightarrow 2^N$

$N-1 \rightarrow 2^{N-1}$

○ $\Theta(1)$	○ $\Theta(\log(\log N))$	○ $\Theta(\log N)$	○ $\Theta((\log N)^2)$	○ $\Theta(\sqrt{N})$
○ $\Theta(N)$	○ $\Theta(N \log N)$	○ $\Theta(N^2)$	○ $\Theta(N^2 \log N)$	○ $\Theta(N^3)$
○ $\Theta(N^3 \log N)$	● $\Theta(2^N)$	○ $\Theta(N!)$	○ $\Theta(N^N)$	○ Infinite loop

Q6.3 (3 points) What is the runtime of `unlucky` in terms of N?

```
public void unlucky(int N) {
    for (int i = 0; i < N; i += 805) {
        System.out.println("karen likes sharetea");
    }
    reducer(N);
    reducer(N);
}
public void reducer(int N) {
    if (N <= 1) {
        return;
    }
    for (int i = 0; i < N; i += 67) {
        System.out.println("karen dislikes bobaflation");
    }
    unlucky(N/2);
}
```

$N \quad N \Big\} 3N$

$\frac{N}{2} \quad \frac{N}{2} \Big\} 3N$

$\frac{N}{2} \quad \frac{N}{2} \quad \frac{N}{2} \quad \frac{N}{2}$

$3 \cdot \log N = 6 \cdot N \cdot N = N^2$

○ $\Theta(1)$	○ $\Theta(\log(\log N))$	○ $\Theta(\log N)$	○ $\Theta((\log N)^2)$	○ $\Theta(\sqrt{N})$
○ $\Theta(N)$	● $\Theta(N \log N)$	○ $\Theta(N^2)$	○ $\Theta(N^2 \log N)$	○ $\Theta(N^3)$
○ $\Theta(N^3 \log N)$	○ $\Theta(2^N)$	○ $\Theta(N!)$	○ $\Theta(N^N)$	○ Infinite loop

Q6.4 (2 points) Say we run Dijkstra's algorithm on a directed graph from source vertex s. If all outgoing edges from s are negative, and all other edges in the graph are nonnegative, then running Dijkstra's algorithm from s will ____ return the correct shortest paths tree.

○ Always ● Sometimes ○ Never

Q6.5 (2 points) Say we have a connected graph with unique edge weights. Running Kruskal's algorithm will ____ output the same tree as running Prim's algorithm from any start vertex.

○ Always ● Sometimes ○ Never

Q6.6 (2 points) Say we have a connected graph with E edges, where E is even. Half of the edges have weight 6, and the other half have weight -7. A minimum spanning tree of this graph will ____ have a total weight ≤ 0.

● Always ○ Sometimes ○ Never

Reminder: Height is defined as the maximum number of *edges* (black and red) from the root to a leaf.

Q6.7 (2 points) What is the minimum and maximum height of a **BST** with 15 nodes?

Minimum: 3

Maximum: 14

Q6.8 (2 points) What is the minimum and maximum height of a **heap** with 15 nodes?

Minimum: 3

Maximum: 3

Q6.9 (3 points) What is the minimum and maximum height of an **LLRB** with 15 nodes?

Minimum: 3

Maximum: 4

Q6.10 (3 points) What is the minimum and maximum height of a **Weighted Quick Union** object (without path compression) with 15 elements, where `isConnected(a, b)` returns true for every pair of items?

Minimum: 1

Maximum: 2

Q7 *Hashing* (3 points)

Consider the `hashCode` function below:

```
public int hashCode() {
    int n = 1;
    for (int i = 0; i < this.length(); i++) {
        n *= ((i + 1) * charToInt(this.charAt(i)));
    }
    return n;
}
```

$1 \cdot (1 \cdot 2) \cdot (2 \cdot 1) = 4$

$1 \cdot (1 \cdot 1) \cdot (2 \cdot 2) = 4$

The `charToInt` function converts `'a'` to 1, `'b'` to 2, etc.

Q7.1 (2 points) Given the string `"ab"`, construct a **different** string (with only lowercase letters) that has the same hash code (without integer overflow). If no such string exists, select "Not possible."

String with same hash code as `"ab"`:

"ba"

○ Not possible

Q7.2 (1 point) True or False: The above `hashCode` function is valid.

● True ○ False

Q8 *Sort City* (15 points)

For Q8.1 to Q8.5, consider the scenario and select the best sorting algorithm **from the provided list**. Assume Quicksort is implemented using shuffling and Tony Hoare style partitioning.

Q8.1 (1.25 points) You need to sort a long array of employees by their birthday. The array is already sorted alphabetically by their names. After sorting by birthday, you want employees with the same birthday to be kept in alphabetical order.

○ Insertion sort ○ Heapsort ● Merge sort ○ Selection sort

Q8.2 (1.25 points) You need to sort a long array of integers, where the integers can be any value in the range [0, 2147483647].

○ Insertion sort ○ Heapsort ○ Merge sort ● Radix sort

Q8.3 (1.25 points) You need to sort a long array of items by their `size` property, and the size is always either `small`, `medium`, or `large`.

○ Insertion sort ○ Heapsort ● Counting sort

○ Quicksort ○ Merge sort

Q8.4 (1.25 points) You need to sort a long array of `Dogs`, which implements `Comparable`. You need the sort to be as fast as possible on real-world computing hardware.

- ○ Insertion sort
- ○ Heapsort
- ○ Counting sort
- ● Quicksort
- ○ Merge sort
- ○ Radix sort

Q8.5 (1 point) You are again sorting a long array of `Dogs`, but the array is already sorted except the last 5 `Dogs`.

- ● Insertion sort
- ○ Heapsort
- ○ Counting sort
- ○ Quicksort
- ○ Merge sort
- ○ Radix sort

Q8.6 (1.5 points) Which of the following sorts uses the asymptotically smallest number of swaps in its worst case? Assume Quicksort uses shuffling and Tony Hoare style partitioning.

- ○ Insertion sort
- ● Heapsort
- ○ Quicksort
- ○ Selection sort

Q8.7 (1.5 points) What are the advantages of using Tony Hoare style partitioning over 3-scan partitioning approach? Select all that apply.

- ■ Runs in fewer seconds
- ■ Uses less memory
- ○ None of the above
- ☐ Better asymptotic worst case
- ☐ Hoare provides stability

Natalia the gnome has developed a sorting algorithm `plantSort` to help out Josh, her gnome friend.

```
public static void plantSort(int[] arr){
    int index = 0;
    int n = arr.length;
    while (index < n) {
        if (index == 0 || arr[index] >= arr[index - 1]) {
            index += 1;
        } else {
            int temp = arr[index];
            arr[index] = arr[index - 1];
            arr[index - 1] = temp;
            index -= 1;
        }
    }
}
```

Q8.8 (2 points) Using the `plantSort` algorithm, how many swaps would it take to sort [4, 3, 7, 2, 8]?

○ 2 ○ 3 ● 4 ○ 6 ○ 7 ○ 8

Q8.9 (3 points) What is the runtime of `plantSort` in terms of N, the length of the array?

Best case:	● $\Theta(N)$	○ $\Theta(N \log N)$	○ $\Theta(N^2)$	○ $\Theta(2^N)$	○ Other
Worst case:	○ $\Theta(N)$	○ $\Theta(N \log N)$	● $\Theta(N^2)$	○ $\Theta(2^N)$	○ Other

Q8.10 (1 point) Is `plantSort` stable? ● Yes ○ No

You have reached the end of the exam! Nothing on this page is worth any credit.

More Skittles

Match each of the exams team members to their response to "What's your favorite Skittles flavor?" :)

Aditya	"they literally all taste the same"
Avik	"Red def"
Josh	"anything but yellow"
Karen	"mines red and purple"
Natalia	"purple for me"
Peyrin	"def green but only when its green apple not lime"
Phoenix	"purple or red"
Rushil	"Zij zijn allemaal niet lekker, eerlijk gezegd"
Vanessa	"i like orange"

Even More Skittles

Now, what's *your* favorite Skittles flavor?

○ Red ○ Orange ○ Yellow ○ Green ● Purple ○ Other ○ None

Care to elaborate?

Artificial Grape is the best, though you never eat skittles one at a time.

Feedback

Leave any feedback, comments, and/or drawings in the box below!

02137